Anisa Aulia Alhaqi/18224080

KAMUS UMUM

```
constant NULL = NIL
type address : pointer to Node
type BinTree : Address
type TreeNode : <     info : ElType,
                      left : BinTree,
                      right : BinTree >
type Node :         <  info : ElType,
                      next : AddressList >
type NodeList : AddressList
```

1. function search(P : BinTree, x : ElType) → boolean
   ALGORITMA
   ```
        depend on (P) :
             (P = NIL) :
                  → false
             (P↑.info = x) :
                  → true
             (P↑.info ≠ x) :
                  → search(P↑.left, x) or search(P↑.right, x)
   ```
2. function isSkewLeft(P : BinTree) → boolean
   ALGORITMA
   ```
        depend on (P) :
             (P = NIL) :
                  → true
             (P↑.right ≠ NIL) :
                  → false
             (P↑.right = NIL) :
                  → isSkewLeft(P↑.left)
   ```
3. function isSkewRight(P : BinTree) → boolean
   ALGORITMA
   ```
        depend on (P) :
             (P = NIL) :
                  → true
             (P↑.left ≠ NIL) :
   ```

```
                    → false
               (P↑.left = NIL) :
                    → isSkewRight(P↑.right)
```

4. `function level (P : BinTree, x : ElType) → integer`

```
ALGORITMA
     depend on (P) :
          (P↑.info = x):
               → 1
          (search(P↑.left, x)):
               → 1 + search(P↑.left, x)
          (search(P↑.right, x)):
               → 1 + search(P↑.right, x)
```

5. `procedure addDaun (input/output P : BinTree, input x, y : ElType, input Kiri : boolean)`

```
KAMUS LOKAL
     found : static boolean
ALGORITMA
     found ← false
     if(P↑.info = x and P↑.left = NIL and P↑.right = NIL) then
          if(Kiri) then
               P↑.kiri ← newTreeNode(y)
          else
               P↑.kanan ← newTreeNode(y)
          found ← true
     else
          if(not(P↑.left = NIL and found)) then
               addDaun(P↑.left, x, y, Kiri)
          if(not(P↑.right = NIL and found)) then
               addDaun(P↑.kanan, x, y, Kiri)
```

6. `procedure delDaun(input/output P : BinTree, input x : ElType)`

```
ALGORITMA
     if(P↑.info = x and P↑.left = NIL and P↑.right = NIL) then
          deallocTreeNode(p)
     else
          if (P↑.left ≠ NIL) then
               delDaun(P↑.left, x)
          if (P↑.right ≠ NIL) then
```

```
delDaun(P↑.right, x)
```